

ECL Thor Roxie comparison

Yes — you're mostly right! Here's a quick check + correction to sharpen your understanding:

✅ Correct points you already made:

- ECL is declarative, easier to learn, used to **describe what** you want, not **how** to do it.
- Thor is for **batch data processing**; uses a **master-slave** setup.
- Roxie is for **real-time querying**, collects partial results and merges them.

🔧 Let's tighten it:

1. Thor:

- Yes, it's **massively parallel**.
- Master builds an **execution graph**, assigns work to slaves.
- Think of it as the “Hadoop” part — **ETL, transformations, big batch jobs**.

2. Roxie:

- Used in **real-time environments** (think search engines or APIs).
- Slaves run queries in parallel, send partial results.
- Roxie merges and **returns the best/combined result quickly**.

3. ECL:

- Used for both **Thor and Roxie**.
- Same language, but **context (Thor or Roxie)** decides whether it's batch or real-time.

Want a mini quiz to lock it in?

Great — that's a solid roadmap! Here's how we can go step-by-step:

1. Basics First

- Activity
- RECORD declaration
- Function + MODULE
- Schema-on-read

2. Files

- ECL file structure
- Importing files
- Spraying and Reading data

3. Data Shaping & Processing

- TRANSFORM
- MODULE / PROJECT
- ITERATE, ROLLUP
- SORT, JOIN, DEDUP
- NORMALIZE / DENORMALIZE
- DISTRIBUTE

4. Execution & Analysis

- Reading Execution Graph
- GROUP with SUM, AVE, COUNT
- TABLE and AGGREGATE

Want to begin with “Activity + RECORD Declaration” and I’ll show examples?

Perfect. Here's a **quick crash course** to get you started on **ECL basics, Activity, and RECORD declaration**.

What is ECL (Enterprise Control Language)?

- A **declarative** language (you say *what* you want, not *how*).
- Used in **HPCC Systems**.
- Written in `.ec1` files and compiled into workunits for Thor (batch) or Roxie (query).
- Syntax is **semicolon-based**, case-insensitive, and similar to SQL + C.

Basic ECL Syntax

ecl

```
MyVar := 100;      // := is the assignment operator
OUTPUT(MyVar);    // Prints value
```

Everything ends with `;`. Think of `:=` as “define this thing”.

What is an Activity in ECL?

An *activity* is a processing step like `OUTPUT`, `SORT`, `JOIN`, `ROLLUP`, etc. These build the **execution graph** in Thor or Roxie.

Example:

```
ecl
```

```
DATA := DATASET([{'Tom', 24}, {'Amy', 28}], {STRING Name; INTEGER Age;});  
SORTED := SORT(DATA, Age);  
OUTPUT(SORTED);
```

→ Activities: `DATASET`, `SORT`, `OUTPUT`.

RECORD Declaration

It defines the **structure/schema** of your data.

Example:

```
ecl
```

```
PERSON := RECORD  
  STRING20 Name;  
  INTEGER2 Age;  
END;
```

This is just like a struct/class — you're saying what each “row” will look like.

Using RECORD with DATASET

```
ecl
```

```
PERSON := RECORD  
  STRING20 Name;  
  INTEGER2 Age;  
END;
```

```
DATA := DATASET([
```

```
{'John', 30},  
{'Mary', 25}  
], PERSON);  
  
OUTPUT(DATA);
```

Ready to move on to **Function Declaration** or want some quick practice/quiz on this?

Here's a **quick practice quiz** to test what you've learned:

Quick ECL Quiz (Answer after all questions)

1. What is the purpose of the `:=` operator in ECL?
2. What keyword is used to define a schema/row structure in ECL?
3. Is ECL case-sensitive?
4. What is the type of the following value: `'Sam'` ?
5. What does this activity do: `OUTPUT(SORT(DATA, Age));`
6. In this RECORD:

```
ecl  
  
RECORD  
  STRING20 Name;  
  INTEGER2 Age;  
END;
```

What's the max character length for Name?

Answers

1. Assignment (define a value or operation).
2. `RECORD`
3. No, ECL is **case-insensitive**.
4. `STRING`
5. Sorts the dataset by `Age` and outputs it.

6. 20 characters.

Shall we continue to **Functions** and **MODULES** next?

Great. Let's dive into **Function Declaration** and **MODULE** in ECL.

Function Declaration in ECL

Used to encapsulate reusable logic.

Syntax:

```
ecl

FunctionName(parameter1_type param1, parameter2_type param2) := FUNCTION
    RETURN (param1 + param2);
END;
```

Example:

```
ecl

AddNums(INTEGER a, INTEGER b) := FUNCTION
    RETURN (a + b);
END;

OUTPUT(AddNums(5, 3)); // Returns 8
```

You can call this like a normal function. It must RETURN something.

MODULE in ECL

Used to **group multiple functions or constants** together.

Syntax:

```
ecl
```

```
MyMath := MODULE
  EXPORT Add(INTEGER a, INTEGER b) := FUNCTION
    RETURN (a + b);
  END;

  EXPORT Multiply(INTEGER x, INTEGER y) := FUNCTION
    RETURN (x * y);
  END;
END;
```

Usage:

ecl

```
OUTPUT(MyMath.Add(4, 6));
OUTPUT(MyMath.Multiply(3, 7));
```

EXPORT means the function can be used outside the module.

Ready to move to **File Handling** (ECL files, Importing, Spraying, Reading)?

Perfect. Let's cover **ECL File Handling: ECL files, Importing, Spraying, Reading a file** — all very important for real datasets.

1. ECL File (.ecl)

Just a plain text file with ECL code. You write your logic, save as `.ecl`, and run it on Thor or Roxie.

2. Importing Other ECL Files

If you split logic into modules/files:

ecl

```
IMPORT MyLib; // Imports code from MyLib.ecl
```

Use `EXPORT` in your module if you want it accessible after import.

3. Spraying a File (HPCC)

Spray = Upload and distribute a local file to the HPCC cluster (Thor).

Usually done from ECL Watch GUI or with a command like:

```
bash  
  
dfuplus action=spray ...
```

It creates a logical filename like:

```
~mydataset::customer_data
```

4. Reading a Sprayed File

You need to define the **RECORD** and use `DATASET` .

Example:

```
ecl  
  
CustomerRec := RECORD  
  STRING20 Name;  
  INTEGER2 Age;  
  STRING20 City;  
END;  
  
Customers := DATASET('~mydataset::customer_data', CustomerRec, CSV);  
  
OUTPUT(Customers);
```

`'csv'` tells ECL it's a CSV file. You can also use `FLAT` , `XML` , `JSON` , etc.

Ready for **Transform**, **Iterate**, and **Rollup** next?

Sure! Here's a complete ECL activity that:

- Imports a **CSV file**
 - Defines a **RECORD structure**
 - Loads the dataset
 - Outputs the **first 10 records**
-

✓ ECL Code Example

ecl

```
// 1. Define the record structure
PersonRec := RECORD
  STRING20 Name;
  INTEGER2 Age;
  STRING20 City;
END;

// 2. Read from a sprayed CSV file
People := DATASET('~mydataset::people_data', PersonRec, CSV);

// 3. Output the first 10 records
OUTPUT(PEOPLE[1..10]);
```

✓ Make sure the file `'~mydataset::people_data'` is **sprayed to your Thor cluster**.

⚠ File should not have a header unless handled separately (use `HEADING(1)` if needed).

Ready for **TRANSFORM, ITERATE, and ROLLUP** next?

Great! Let's break down **TRANSFORM, ITERATE, and ROLLUP** — core ECL activities for **data shaping**.

🔄 1. TRANSFORM

Used to define how a **record should be modified or created**.

Example: Add 5 to Age

```
ecl

PersonRec := RECORD
  STRING20 Name;
  INTEGER2 Age;
END;

People := DATASET('~mydataset::people_data', PersonRec, CSV);

AddFive := TRANSFORM(PersonRec)
  SELF.Age := LEFT.Age + 5;
  SELF.Name := LEFT.Name;
END;

Modified := PROJECT(People, AddFive);
OUTPUT(Modified[1..5]);
```

2. ITERATE

Used to **process records one-by-one** and **maintain a running total/state**.

Example: Total age of people

```
ecl

Rec := RECORD
  INTEGER2 Age;
END;

DATA := DATASET([ {25}, {30}, {20} ], Rec);

SumRec := RECORD
  INTEGER TotalAge;
END;

SumFn := TRANSFORM(SumRec, SumRec Right, Rec Left)
  SELF.TotalAge := Right.TotalAge + Left.Age;
END;
```

```
Result := ITERATE(DATA, SumFn, TRANSFORM(SumRec, SELF.TotalAge := 0));  
OUTPUT(Result);
```

3. ROLLUP

Groups sorted records and performs **aggregation** (e.g., summing salaries in same department).

Example: Sum age by City

```
ecl  
  
PersonRec := RECORD  
  STRING20 City;  
  INTEGER2 Age;  
END;  
  
DATA := DATASET([  
  {'NY', 25}, {'NY', 30}, {'LA', 28}, {'LA', 22}  
], PersonRec);  
  
AggRec := RECORD  
  STRING20 City;  
  INTEGER TotalAge;  
END;  
  
AggFn := TRANSFORM(AggRec, PersonRec L, PersonRec R)  
  SELF.City := L.City;  
  SELF.TotalAge := L.Age + R.Age;  
END;  
  
Sorted := SORT(DATA, City);  
Grouped := ROLLUP(Sorted, City, AggFn);  
OUTPUT(Grouped);
```

Ready to continue with **SORT, JOIN, DEDUP?**

Awesome — let's now cover **SORT, JOIN, and DEDUP** in ECL, all used to **organize and relate data**.

1. SORT

Used to **order records** by one or more fields.

Example:

```
ecl

PersonRec := RECORD
  STRING20 Name;
  INTEGER2 Age;
END;

People := DATASET('~mydataset::people_data', PersonRec, CSV);

Sorted := SORT(People, Age); // Ascending
OUTPUT(Sorted[1..5]);
```

Use `SORT(People, -Age)` for descending.

2. JOIN

Combines two datasets based on a **common field**.

Example: Join people with cities

```
ecl

PersonRec := RECORD
  STRING20 Name;
  STRING20 City;
END;

CityRec := RECORD
  STRING20 City;
  STRING20 State;
END;
```

```
People := DATASET('~mydataset::people_data', PersonRec, CSV);
```

```
Cities := DATASET('~mydataset::city_info', CityRec, CSV);
```

```
JoinRec := RECORD
```

```
  STRING20 Name;
```

```
  STRING20 City;
```

```
  STRING20 State;
```

```
END;
```

```
JoinFn := TRANSFORM(JoinRec)
```

```
  SELF.Name := LEFT.Name;
```

```
  SELF.City := LEFT.City;
```

```
  SELF.State := RIGHT.State;
```

```
END;
```

```
Result := JOIN(People, Cities, LEFT.City = RIGHT.City, JoinFn);
```

```
OUTPUT(Result[1..5]);
```

3. DEDUP

Removes **duplicate records** based on one or more fields.

Example: Keep only 1 record per name

```
ecl
```

```
UniquePeople := DEDUP(People, Name);
```

```
OUTPUT(UniquePeople[1..5]);
```

It keeps the **first occurrence** based on dataset order.

Shall we move to **Normalize / Denormalize** and **Distribute** next?

Great! Let's now go through **NORMALIZE**, **DENORMALIZE**, and **DISTRIBUTE** — all used to reshape and optimize data distribution.

1. NORMALIZE

Turns **one record into many** (i.e., splits columns or lists into rows).

Example: Split comma-separated string into multiple rows

```
ecl

Rec := RECORD
  STRING Names;
END;

DATA := DATASET(['Tom,Amy,Joe'], Rec);

SplitRec := RECORD
  STRING Name;
END;

SplitFn := TRANSFORM(SplitRec)
  SELF.Name := TRIM(FIELD);
END;

Result := NORMALIZE(DATA, 3, SplitFn, CSV(Names));
OUTPUT(Result);
```

You can also use `NORMALIZE(dataset, count, transform_fn)` when you already know how many splits are needed.

2. DENORMALIZE

Turns **many records into one** (i.e., aggregate rows into a single record).

Example: Group all names by City

```
ecl

Rec := RECORD
  STRING20 City;
  STRING20 Name;
END;

DATA := DATASET([
```

```

{'NY', 'Tom'},
{'NY', 'Amy'},
{'LA', 'Joe'}
], Rec);

AggRec := RECORD
  STRING20 City;
  STRING100 AllNames;
END;

AggFn := TRANSFORM(AggRec, Rec L, Rec R)
  SELF.City := L.City;
  SELF.AllNames := L.Name + ',' + R.Name;
END;

Sorted := SORT(DATA, City);
Result := DENORMALIZE(Sorted, City, AggFn);
OUTPUT(Result);

```

3. DISTRIBUTE

Physically redistributes data across nodes **based on a field** (used before JOINS or heavy operations).

Example:

```

ecl

DistPeople := DISTRIBUTE(People, HASH32(Name));

```

Ensures all records with the same `Name` go to the same node — useful for parallel processing.

Next: Want to cover **Execution Graph**, **GROUP**, and **TABLE/AGGREGATE** ?

Perfect. Now let's wrap up with **Execution Graph**, **GROUP**, and **TABLE / AGGREGATE** .

Execution Graph (Thor)

- Generated when ECL is compiled.
- Shows **activities (SORT, JOIN, etc.) as nodes**, connected by data flow.
- Master assigns each **activity to slaves** based on this graph.
- View in **ECL Watch → Workunit → Graph**.

 Use it to debug, see performance bottlenecks, and optimize.

GROUP

Used to **logically group records** for aggregation.

Example: Sum age per city

```
ecl

Grouped := GROUP(People, City);

OUTPUT(Grouped);
```

Usually paired with **ROLLUP**, **ITERATE**, or **AGGREGATE**.

TABLE and AGGREGATE

TABLE

Like SQL's GROUP BY + SELECT. Returns grouped rows with calculated fields.

Example:

```
ecl

AggTable := TABLE(People, {City, AVG(Age) AS AvgAge}, City);

OUTPUT(AggTable);
```

AGGREGATE

Same logic, used for **more complex transformations** with a **TRANSFORM**.

Example:

ecl

```
AggRec := RECORD
```

```
  STRING20 City;
```

```
  REAL AvgAge;
```

```
END;
```

```
AggFn := TRANSFORM(AggRec)
```

```
  SELF.City := LEFT.City;
```

```
  SELF.AvgAge := AVE(GROUP, LEFT.Age);
```

```
END;
```

```
AggData := AGGREGATE(People, AggRec, AggFn, GROUP BY City);
```

```
OUTPUT(AggData);
```

You're now familiar with all **core ECL building blocks** 💡

Want a quick **revision summary** or practice questions now?

Here's a **quick ECL Summary** of all that you've learned:

Basics

- `:=` is assignment
 - `RECORD` defines schema
 - `DATASET(..., RECORD, CSV)` reads data
 - `OUTPUT()` displays results
 - ECL is **declarative**, case-insensitive
-

Functions & Modules

- `FUNCTION` returns logic
 - `MODULE` groups reusable functions
 - `EXPORT` makes them accessible
-

Files & Import

- `.ec1` = ECL script
 - `IMPORT MyLib;` to reuse code
 - **Spray** = upload data to Thor
 - `DATASET('~file', Rec, CSV)` to read sprayed file
-

Data Transformation

- `TRANSFORM` changes each record
 - `PROJECT(dataset, transformFn)` applies transform
-

Iterate & Rollup

- `ITERATE` = row-by-row processing with state
 - `ROLLUP` = group consecutive rows and reduce
-

Sort, Join, Dedup

- `SORT(data, field)`
 - `JOIN(A, B, A.key = B.key, transformFn)`
 - `DEDUP(data, field)` removes duplicates
-

Normalize / Denormalize

- `NORMALIZE` = 1 record → many
 - `DENORMALIZE` = many → 1
-

Distribute

- `DISTRIBUTE(data, HASH32(key))` for better parallelism
-

Execution Graph

- Shows flow of activities in Thor
- Used for debugging + performance



GROUP / TABLE / AGGREGATE

- `GROUP(data, field)`
 - `TABLE(data, {field, SUM(x)}, field)`
 - `AGGREGATE(data, Rec, transformFn, GROUP BY field)`
-

Want a small quiz or hands-on task next?